

Classic 8 queen problem

Name: Hemalakshmi H (192571049)

Aim:

To develop a Python program that solves the 8-Queens Problem using the Backtracking algorithm, ensuring that no two queens attack each other on the same row, column, or diagonal.

Algorithm:

Step 1: Start with an empty 8×8 chessboard. **Step 2:** Place a queen in the first row and first column. **Step 3:** Move to the next row and try placing a queen in a safe column. **Step 4:** Check if the position is safe (no other queen in same column or diagonal). **Step 5:**

- If safe → place the queen and move to the next row.
- If not safe → move to the next column. **Step 6:**
- If no column is safe → backtrack to the previous row and move that queen to the next column. **Step 7:** Repeat until all 8 queens are placed safely. **Step 8:** Display all valid solutions.

Source Code:

```
# Classic 8-Queens problem using backtracking
```

```
N = 8 # size of board and number of queens
```

```
def is_safe(board, row, col):
```

```
    """
```

```
    Check if placing a queen at (row, col) is safe  
    given the current partial board.
```

```
    board[i] = column of queen in row i
```

```
    """
```

```
    for i in range(row):
```

```
        # same column
```

```
        if board[i] == col:
```

```
            return False
```

```
        # same diagonal
```

```
        if abs(board[i] - col) == abs(i - row):
```

```
        return False
    return True
```

```
def solve_n_queens(n=N):
    solutions = []
```

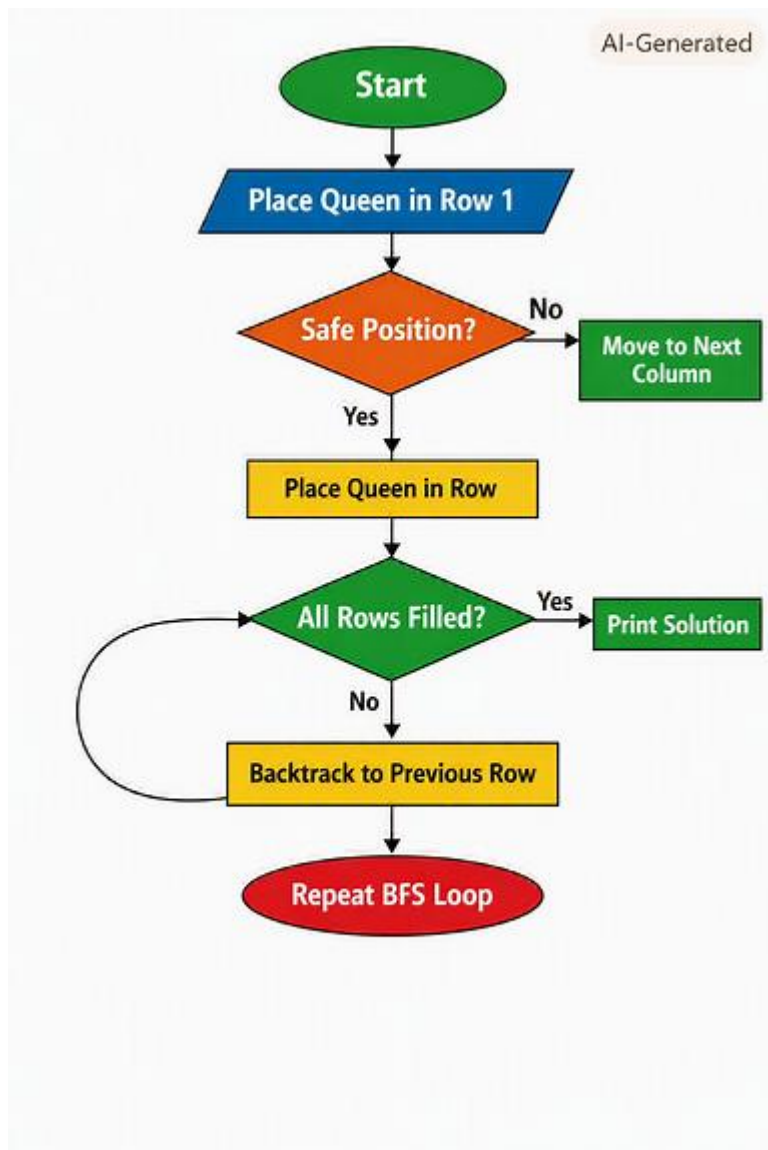
```
    def backtrack(board, row):
        if row == n:
            # found a full valid placement
            solutions.append(board[:])
            return
        for col in range(n):
            if is_safe(board, row, col):
                board[row] = col
                backtrack(board, row + 1)
                board[row] = -1 # reset (backtrack)
```

```
    backtrack([-1] * n, 0)
    return solutions
```

```
def print_board(solution):
    n = len(solution)
    for r in range(n):
        line = ""
        for c in range(n):
            line += "Q " if solution[r] == c else ". "
        print(line)
    print("-" * (2 * n))
```

```
if __name__ == "__main__":  
    sols = solve_n_queens()  
    print(f"Total solutions: {len(sols)}")  
    print("First solution:")  
    print_board(sols[0])
```

Flowchart:



Output:

```
>>>
===== RESTART: C:\Users\hemal\OneDr
Total solutions: 92
First solution:
Q . . . . .
. . . . Q . .
. . . . . Q
. . . . . Q
. . . . Q .
. . Q . . .
. . . . . Q
. Q . . . .
. . . Q . .
-----
```

Result:

The 8-Queens program successfully placed eight queens on the chessboard such that no two queens attacked each other horizontally, vertically, or diagonally. The backtracking algorithm systematically explored all possible configurations and discarded unsafe placements through recursive backtracking.

The program displayed the total number of valid solutions (92) and printed one correct board arrangement as output. Each queen was positioned safely, demonstrating the efficiency of the backtracking approach in solving constraint satisfaction problems.

Thus, the **8-Queens Problem** was solved successfully using the **Backtracking algorithm**.